

Okayish Chat

Project Report — Web Application Development
WAD Course · April 2026

Stack	Python · FastAPI · PostgreSQL · Redis · llama-cpp-python
UI Mode	SPA — Single-Page Application
Architecture	MCS — Model · Controller · Service
Auth	JWT access tokens + Redis-backed opaque refresh tokens (30-day TTL)
Database	PostgreSQL via Docker · Alembic migrations
LLM	Local GGUF model via llama-cpp-python (CPU inference, ChatML prompt)
Bonus	Auto-refresh · Token rotation

1. Architecture Overview

Okayish Chat is a **Single-Page Application** served as static HTML from FastAPI's StaticFiles mount. The backend follows **MCS (Model–Controller–Service)**: routers are thin wrappers, all business logic lives in service modules.

Layer	Location	Responsibility
Model	app/models/	SQLAlchemy ORM definitions, Alembic-managed migrations
Controller	app/controllers/	FastAPI routers — parse requests, delegate to services, return responses
Service	app/services/	All business logic: auth, JWT, Redis sessions, LLM calls, DB queries

JWT + Redis Session Flow

Every protected route passes through **dependencies.py**, which validates the Bearer JWT (HS256, 30-minute expiry) and loads the matching User. Refresh tokens are **opaque random strings** stored in Redis under `session:<token>` with a 30-day TTL. On each refresh, the old token is deleted and a new one issued (**token rotation**).

Step	Action
1	Client sends credentials → POST /auth/login or /auth/register
2	Server returns { access_token (JWT, 30 min), refresh_token (opaque, 30 days) }
3	Client stores both in localStorage; attaches access_token as Authorization: Bearer
4	On 401, client calls POST /auth/refresh with the refresh_token
5	Server verifies in Redis, deletes old token, issues fresh pair (rotation)
6	GitHub OAuth users follow the same path — tokens issued in /callback redirect

2. API Structure

Built with FastAPI, auto-documented at <http://localhost:8000/docs>. Routes marked ■ require a valid JWT Bearer token.

Auth · /auth

Method	Endpoint	Description	Auth
POST	/auth/register	Register new user, returns token pair	—
POST	/auth/login	Login with username + password	—
POST	/auth/refresh	Rotate refresh token, return new pair	—
GET	/auth/github	Redirect to GitHub OAuth authorization page	—
GET	/auth/github/callback	Handle GitHub callback, issue tokens to SPA	—
GET	/auth/me	Return current authenticated user profile	■ JWT

Chats · /chats

Method	Endpoint	Description	Auth
GET	/chats	List all chats for the current user	■ JWT
POST	/chats	Create a new chat thread	■ JWT
DELETE	/chats/{chat_id}	Delete chat and all its messages (cascade)	■ JWT

Messages · /chats/{chat_id}/messages

Method	Endpoint	Description	Auth
GET	/chats/{chat_id}/messages	Retrieve full message history for a chat	■ JWT
POST	/chats/{chat_id}/messages	Send message → LLM generates reply → both saved	■ JWT

Pydantic Schemas

Schema	Fields
UserRegister	username, email (optional), password
UserLogin	username, password
UserOut	id, username, email, created_at
TokenResponse	access_token, refresh_token
RefreshRequest	refresh_token
ChatCreate	title
ChatOut	id, user_id, title, created_at
MessageSend	content
MessageOut	id, chat_id, role, content, created_at

3. Code Structure

Organised as a standard Python package under **app/**, split into the three MCS layers. Frontend files in **frontend/** are served by FastAPI's StaticFiles mount.

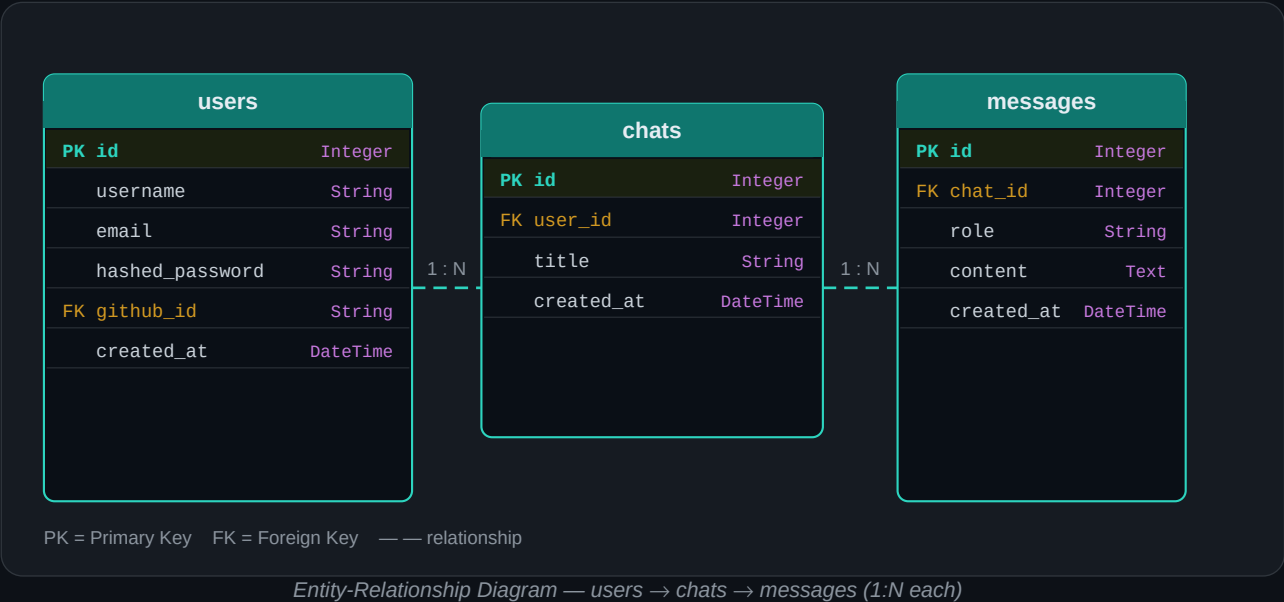
okayish_chat/ ■ ■ ■ app/ ■ ■ ■ ■ controllers/ ■ ■ ■ ■ auth.py ■ ■ ■ ■ chat.py ■ ■ ■ ■ message.py ■ ■ ■ ■ models/ ■ ■ ■ ■ user.py ■ ■ ■ ■ chat.py ■ ■ ■ ■ message.py ■ ■ ■ ■ schemas/ ■ ■ ■ ■ services/ ■ ■ ■ ■ auth_service.py ■ ■ ■ ■ chat_service.py ■ ■ ■ ■ message_service.py ■ ■ ■ ■ llm_service.py ■ ■ ■ ■ config.py ■ ■ ■ ■ database.py ■ ■ ■ ■ dependencies.py ■ ■ ■ ■ redis_client.py ■ ■ ■ ■ main.py ■ ■ ■ frontend/ ■ ■ ■ ■ index.html ■ ■ ■ ■ login.html ■ ■ ■ ■ register.html ■ ■ ■ ■ chat.html ■ ■ ■ alembic/versions/ ■ ■ ■ .env ■ ■ ■ requirements.txt ■ ■ ■ model.gguf	<table><tr><td>main.py</td><td>App entry point — registers all routers, mounts /static, serves redirect.</td></tr><tr><td>controllers/auth.py</td><td>Registration, login, token refresh, GitHub OAuth callback, /me.</td></tr><tr><td>controllers/chat.py</td><td>List, create and delete chat threads.</td></tr><tr><td>controllers/message.py</td><td>Retrieve message history and send messages to the LLM.</td></tr><tr><td>services/auth_service.py</td><td>bcrypt hashing, JWT encode/decode, Redis session management, GitHub user upsert.</td></tr><tr><td>services/llm_service.py</td><td>Lazy-loads GGUF model on first request (singleton). Exposes generate_response() and stream_response().</td></tr><tr><td>services/message_service.py</td><td>Saves user message, calls LLM, saves reply, returns both.</td></tr><tr><td>services/chat_service.py</td><td>DB CRUD for the Chat model.</td></tr><tr><td>database.py</td><td>SQLAlchemy engine + session factory + declarative Base.</td></tr><tr><td>redis_client.py</td><td>Single Redis connection from REDIS_URL env var.</td></tr><tr><td>config.py</td><td>pydantic-settings — reads all secrets from .env file.</td></tr><tr><td>dependencies.py</td><td>FastAPI Depends() — validates Bearer JWT, loads User from DB.</td></tr></table>	main.py	App entry point — registers all routers, mounts /static, serves redirect.	controllers/auth.py	Registration, login, token refresh, GitHub OAuth callback, /me.	controllers/chat.py	List, create and delete chat threads.	controllers/message.py	Retrieve message history and send messages to the LLM.	services/auth_service.py	bcrypt hashing, JWT encode/decode, Redis session management, GitHub user upsert.	services/llm_service.py	Lazy-loads GGUF model on first request (singleton). Exposes generate_response() and stream_response().	services/message_service.py	Saves user message, calls LLM, saves reply, returns both.	services/chat_service.py	DB CRUD for the Chat model.	database.py	SQLAlchemy engine + session factory + declarative Base.	redis_client.py	Single Redis connection from REDIS_URL env var.	config.py	pydantic-settings — reads all secrets from .env file.	dependencies.py	FastAPI Depends() — validates Bearer JWT, loads User from DB.
main.py	App entry point — registers all routers, mounts /static, serves redirect.																								
controllers/auth.py	Registration, login, token refresh, GitHub OAuth callback, /me.																								
controllers/chat.py	List, create and delete chat threads.																								
controllers/message.py	Retrieve message history and send messages to the LLM.																								
services/auth_service.py	bcrypt hashing, JWT encode/decode, Redis session management, GitHub user upsert.																								
services/llm_service.py	Lazy-loads GGUF model on first request (singleton). Exposes generate_response() and stream_response().																								
services/message_service.py	Saves user message, calls LLM, saves reply, returns both.																								
services/chat_service.py	DB CRUD for the Chat model.																								
database.py	SQLAlchemy engine + session factory + declarative Base.																								
redis_client.py	Single Redis connection from REDIS_URL env var.																								
config.py	pydantic-settings — reads all secrets from .env file.																								
dependencies.py	FastAPI Depends() — validates Bearer JWT, loads User from DB.																								

LLM Integration

llm_service.py uses llama-cpp-python to run a GGUF model entirely on CPU. The model is loaded once as a module-level singleton on first request. The ChatML prompt template is used with stop tokens to prevent runaway generation. The service also exposes stream_response(), a generator yielding tokens via stream=True for future SSE integration.

4. Database Schema

PostgreSQL is the primary database, managed via **Alembic** migrations (initial revision d6dd5b713066). Three tables, two one-to-many relationships.



users

Column	Type	Nulla ble	Notes
id	Integer	NO	Primary key, auto-increment, indexed
username	String	NO	Unique, indexed — the login identifier
email	String	YES	Unique, indexed — optional for password accounts
hashed_password	String	YES	bcrypt hash — NULL for GitHub-only accounts
github_id	String	YES	Unique GitHub user ID — NULL for local accounts
created_at	DateTime	YES	UTC timestamp, defaults to now()

chats

Column	Type	Nulla ble	Notes
id	Integer	NO	Primary key, auto-increment, indexed
user_id	Integer	NO	FK → users.id, cascade delete

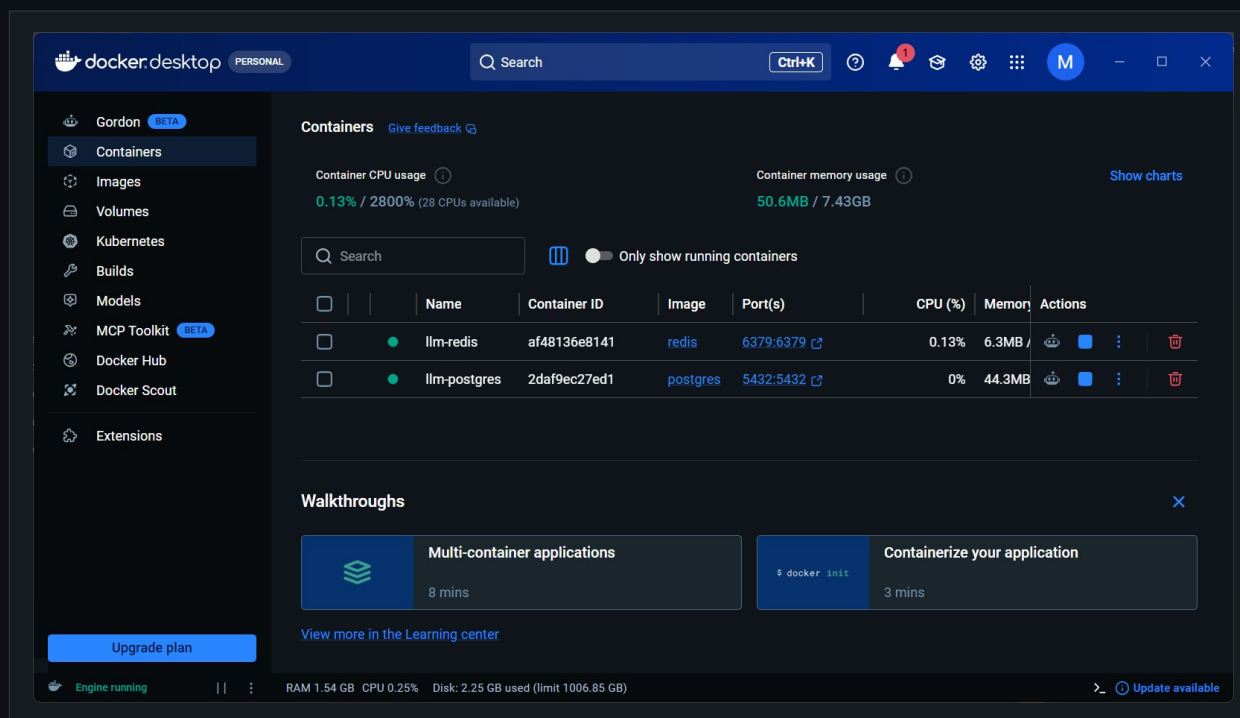
Column	Type	Nullable	Notes
title	String	YES	Auto-renamed from the first user message
created_at	DateTime	YES	UTC timestamp, defaults to now()

messages

Column	Type	Nullable	Notes
id	Integer	NO	Primary key, auto-increment, indexed
chat_id	Integer	NO	FK → chats.id, cascade delete
role	String	NO	Either 'user' or 'assistant'
content	Text	NO	Full message body (no length limit)
created_at	DateTime	YES	UTC timestamp, defaults to now()

5. Docker Setup

PostgreSQL and Redis run as Docker containers so no local installation is required. Docker Desktop must be open with the **Engine running** (green dot, bottom-left) before starting the application.



Docker Desktop showing both containers running — llm-redis (6379) and llm-postgres (5432)

Starting the containers

Run these two commands once to create the containers. On subsequent restarts use `docker start` instead of `docker run`.

```
# Start PostgreSQL (first time only)
docker run -d --name llm-postgres \ -e POSTGRES_USER=postgres \ -e
POSTGRES_PASSWORD=password \ -e POSTGRES_DB=llm_chat \ -p 5432:5432 postgres

# Start Redis (first time only)
docker run -d --name llm-redis -p 6379:6379 redis
```

After every PC restart

```
# Restart both stopped containers
docker start llm-postgres docker start llm-redis

# Verify both are running
docker ps
```

What each container does

Container	Image	Port	Purpose
llm-postgres	postgres	5432	Primary database — stores users, chats, and messages
llm-redis	redis	6379	Session store — holds refresh tokens with 30-day TTL

Environment (.env) configuration

Create a `.env` file in the project root (copy from `.env.example`) and fill in the values below. The `DATABASE_URL` must match the docker run credentials above.

Variable	Example / Description
<code>DATABASE_URL</code>	<code>postgresql://postgres:password@localhost:5432/llm_chat</code>
<code>REDIS_URL</code>	<code>redis://localhost:6379</code>
<code>SECRET_KEY</code>	Generate with: <code>python -c "import secrets; print(secrets.token_hex(32))"</code>
<code>GITHUB_CLIENT_ID</code>	From your GitHub OAuth App settings page
<code>GITHUB_CLIENT_SECRET</code>	From your GitHub OAuth App settings page
<code>GITHUB_REDIRECT_URI</code>	<code>http://localhost:8000/auth/github/callback</code>

Full startup sequence

```
# 1. Start Docker containers
docker start llm-postgres && docker start llm-redis

# 2. Activate virtual environment
venv\Scripts\activate # Windows source venv/bin/activate # macOS / Linux

# 3. Run database migrations (first time or after schema changes)
alembic upgrade head

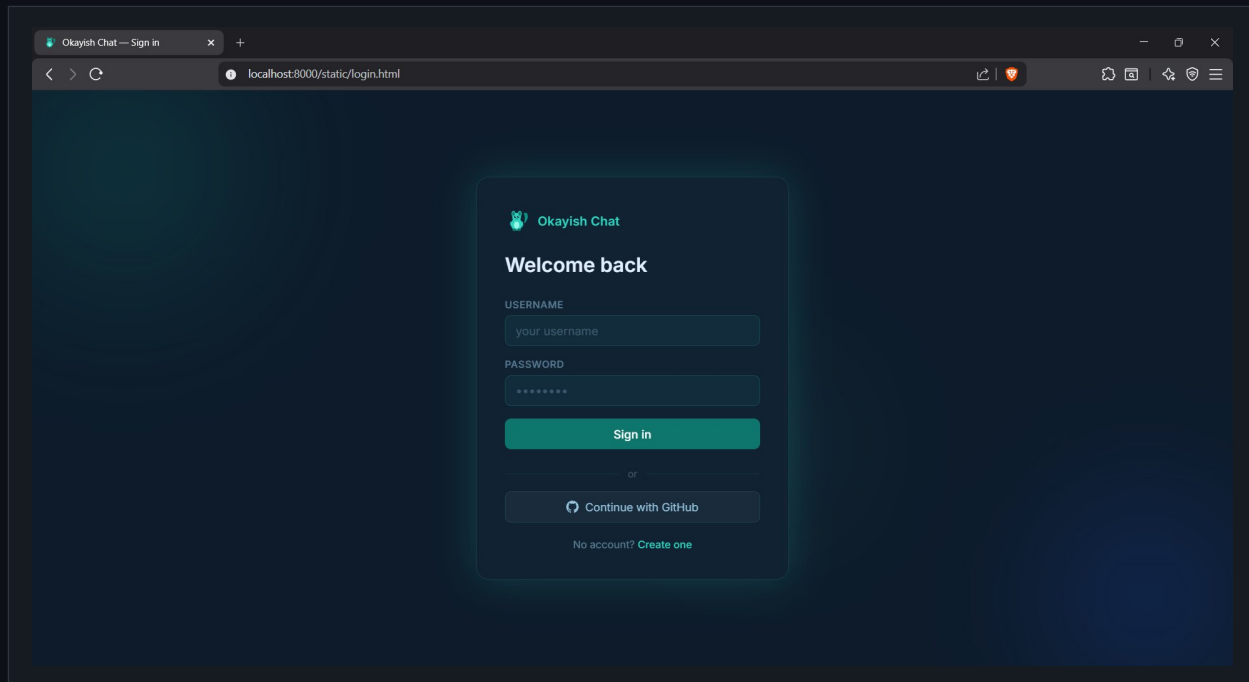
# 4. Start the app
uvicorn app.main:app --reload

# 5. Open in browser
http://localhost:8000
```

6. User Interface

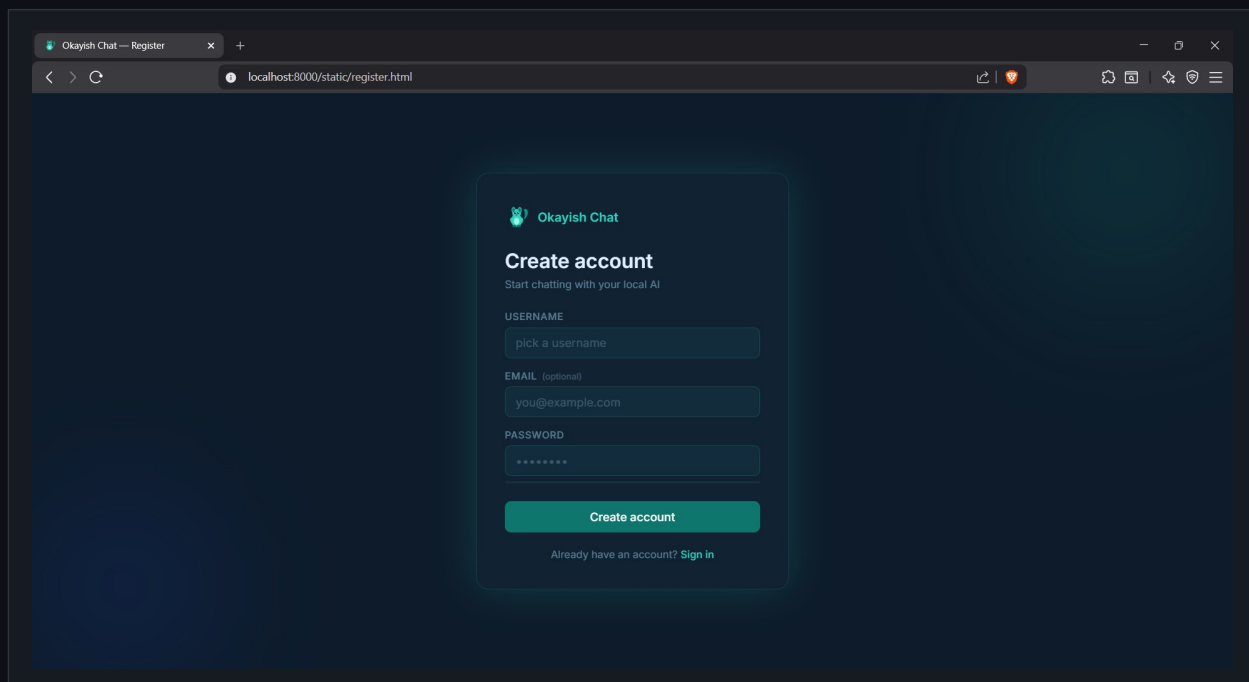
The SPA frontend consists of four pages served from `/static`. Chat threads appear as tabs across the top bar; the active tab is highlighted in teal.

Sign In



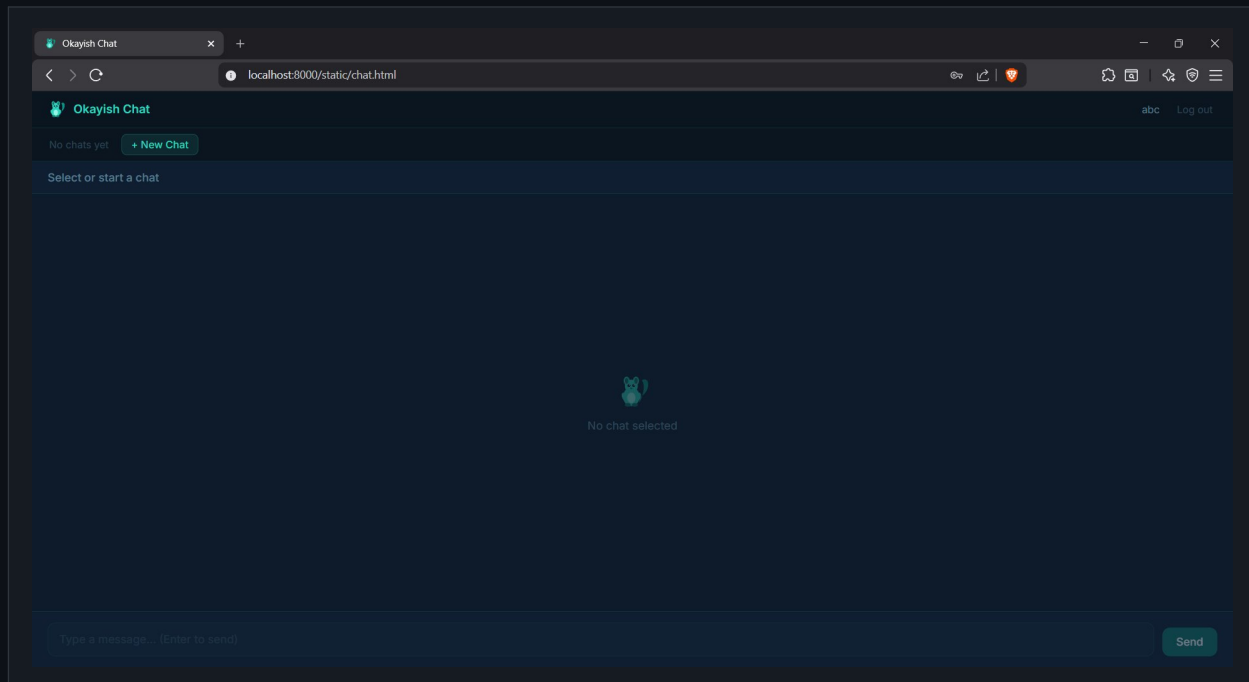
Login page — username/password form with GitHub OAuth option

Register



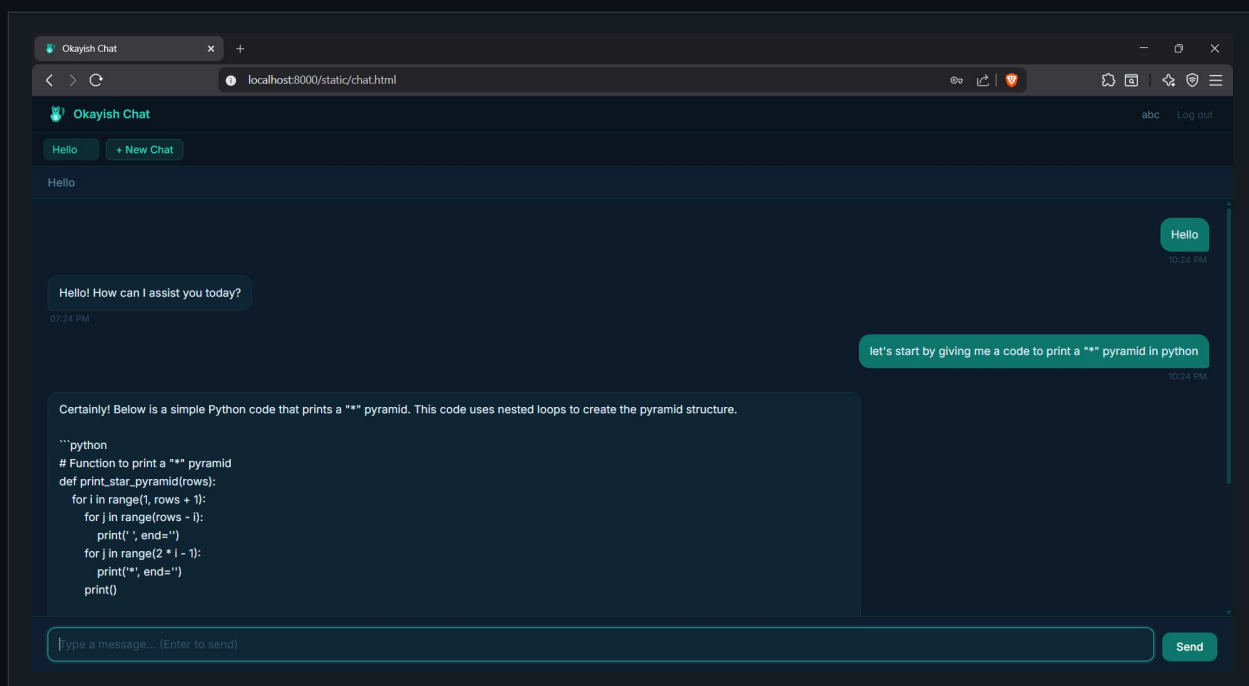
Registration page — username required, email optional

Chat — Empty State



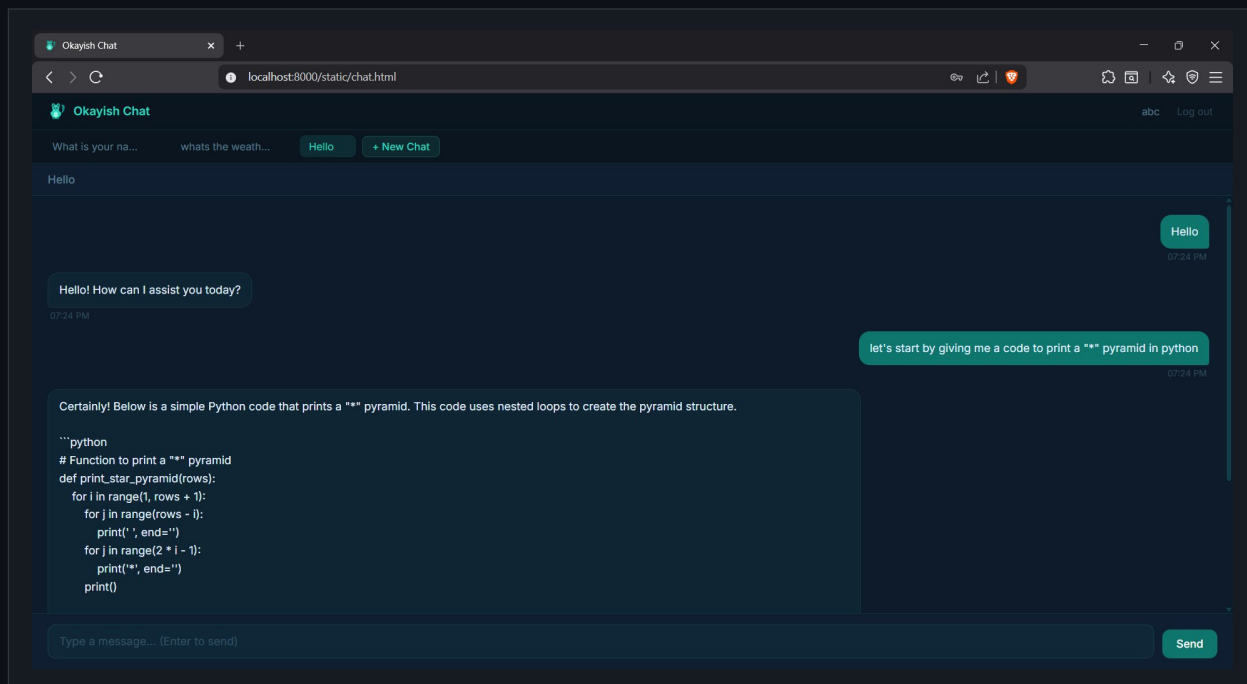
Chat page after login, no chats created yet

Chat — Active Conversation



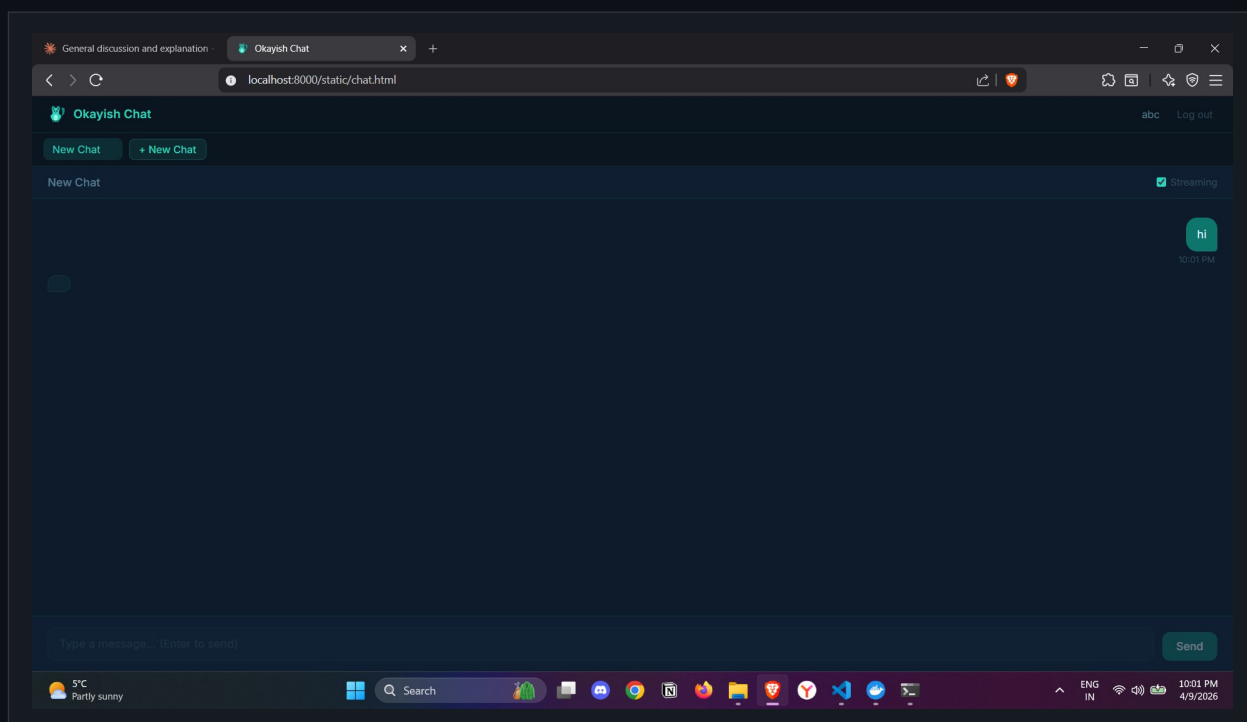
Active chat — user message on the right, LLM reply on the left

Multiple Chat Tabs



Multiple threads visible in the top tab bar; active tab highlighted

Chat — In Use



Chat page during an active session

7. API Documentation

FastAPI auto-generates interactive OpenAPI docs at <http://localhost:8000/docs>.

Group	Method	Path	Summary
auth	POST	/auth/register	Register
auth	POST	/auth/login	Login
auth	POST	/auth/refresh	Refresh
auth	GET	/auth/github	Github Login
auth	GET	/auth/github/callback	Github Callback
auth	GET	/auth/me	Me ■
chats	GET	/chats	List Chats ■
chats	POST	/chats	Create Chat ■
chats	DELETE	/chats/{chat_id}	Delete Chat ■
messages	GET	/chats/{chat_id}/messages	Get Messages ■
messages	POST	/chats/{chat_id}/messages	Send Message ■
default	GET	/	Root (redirect)

8. Bonus Features Implemented

Two of the three bonus requirements from the specification are implemented.

★ Auto-refresh of the chat view

`startAutoRefresh()` in `chat.html` polls the active chat's messages endpoint every 15 seconds. If the message count changes — e.g. from another session — the view re-renders silently. A small `■` 'refreshing' indicator appears in the header during the poll. This matches the spec note that 'periodically reloading the page to refresh the chat view is acceptable'.

★ Refresh token rotation

Every call to `POST /auth/refresh` invalidates the old token in Redis with `r.delete()` before issuing a new one. This means a stolen refresh token becomes useless the moment the legitimate client uses it next — a security practice that goes beyond the base requirement of simply storing tokens with a TTL.

Soft deadline: 14 May 2026 · Hard deadline: 18 May 2026 · Submitted via GitHub